

Teoría de Algoritmos
(75.29) Curso Echevarría

Trabajo Práctico 0
Cuartetos de Números Primos

2do Cuatrimestre 2026

Facundo Beloqui
112570

Índice

1. Supuestos **3**

2. Diseño **3**

3. Análisis de Complejidad **4**

4. Tiempos de Ejecución **4**

5. Informe de Resultados **5**

1. Supuestos

Para el desarrollo y análisis del presente algoritmo modificado, se han establecido los siguientes supuestos y condiciones:

- **Formación de Cuartetos:** Un cuarteto de números primos en la misma decena sólo puede ocurrir si los primos terminan en 1, 3, 7 y 9 (por ejemplo, 11, 13, 17, 19). Cualquier otra combinación incluiría un número par o un múltiplo de 5, los cuales no son primos (a excepción del 2 y el 5, que no forman cuartetos en la misma decena).
- **Salto de Iteración (Step):** Basado en el supuesto anterior, el primer número de un posible cuarteto siempre debe terminar en 1. Por lo tanto, se puede iterar de a decenas comenzando desde el 11 (es decir, 11, 21, 31, 41...), reduciendo drásticamente el espacio de búsqueda.
- **Optimización de Primalidad:** Para determinar si un número N es primo, es suficiente buscar divisores hasta la raíz cuadrada de N ($\sqrt{N}$). Si N tiene un divisor mayor que $\sqrt{N}$, necesariamente debe tener un divisor menor o igual a $\sqrt{N}$.

2. Diseño

El algoritmo se compone de una función auxiliar para verificar la primalidad y una función principal que orquesta la búsqueda.

Pseudocódigo

```
FUNCION esprimo(n):
    i = 2
    MIENTRAS i <= RAIZ_CUADRADA(n):
        SI n MOD i == 0 ENTONCES
            RETORNAR Falso
        FIN SI
        i = i + 1
    FIN MIENTRAS
    RETORNAR Verdadero

FUNCION prog():
    t1 = OBTENER_TIEMPO_ACTUAL()

    PARA i DESDE 11 HASTA 1000000 CON PASO 10:
        SI esprimo(i) Y esprimo(i + 2) Y esprimo(i + 6) Y esprimo(i + 8) ENTONCES
            IMPRIMIR i, i + 2, i + 6, i + 8
        FIN SI
    FIN PARA

    t2 = OBTENER_TIEMPO_ACTUAL()
    IMPRIMIR (t2 - t1)
```

Estructuras de Datos

El programa modificado, al igual que el original, no requiere el uso de estructuras de datos complejas (como listas, arreglos o grafos). Únicamente se emplean **variables de tipo primitivo** (enteros y flotantes) para el control de iteraciones, cálculos aritméticos y medición de tiempo.

3. Análisis de Complejidad

Algoritmo Original

La función `esprimo(n)` original itera desde $i = 2$ hasta $n - 1$. Esto implica $\mathcal{O}(N)$ operaciones por cada llamado, ya que no se detiene al encontrar un divisor ni limita su búsqueda a la raíz cuadrada.

La función principal `prog()` itera $M = 1{,}000{,}000$ veces (de a 1 paso). En el peor caso, para cada número se llama a la función `esprimo(n)` 4 veces. La complejidad temporal total del algoritmo original es del orden de $\mathcal{O}(M^2)$.

Algoritmo Modificado

La función `esprimo(n)` modificada itera sólo hasta $\sqrt{N}$ y retorna `False` inmediatamente al hallar un divisor. Su peor caso (cuando n es primo) es $\mathcal{O}(\sqrt{N})$.

La función principal iterará desde 11 hasta $M = 1{,}000{,}000$ con saltos de 10. Esto reduce la cantidad de iteraciones a $M/10$ (lo cual sigue siendo $\mathcal{O}(M)$ en notación asintótica, pero con una constante multiplicativa 10 veces menor). Para cada iteración, se realizan a lo sumo 4 llamadas a `esprimo(n)`.

Por lo tanto, la complejidad temporal del algoritmo modificado se reduce significativamente a $\mathcal{O}(M\sqrt{M})$.

4. Tiempos de Ejecución

Se realizaron mediciones ejecutando el algoritmo modificado para distintos rangos máximos. Los resultados se resumen en la siguiente tabla y gráfico:

Rango (N)	**Tiempo de Ejecución (segundos)**
200.000	0.4263
400.000	1.1150
600.000	1.9896
800.000	3.1063
1.000.000	4.1253

Cuadro 1: Tiempos de ejecución empíricos del algoritmo modificado.

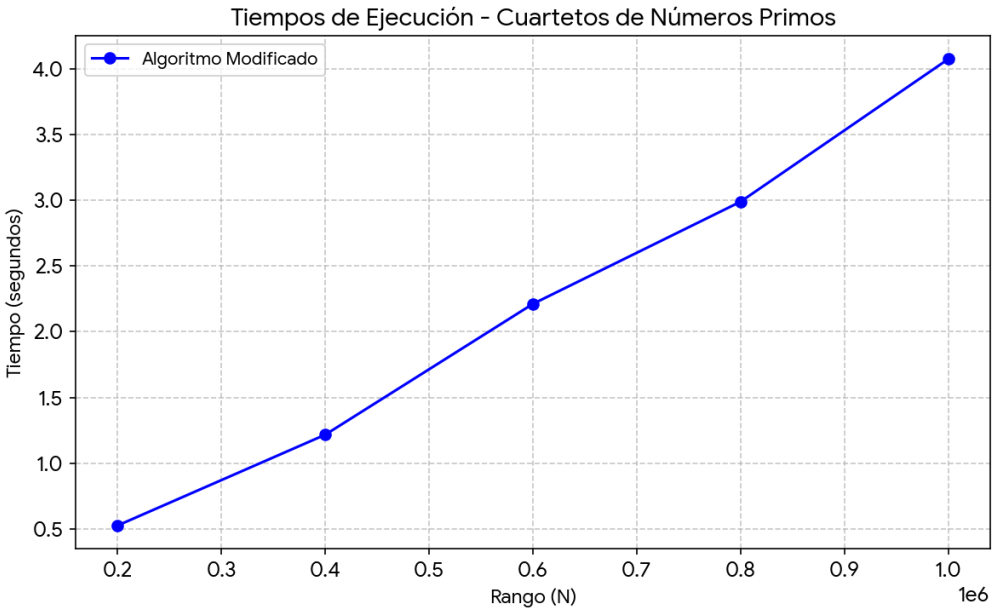


Figura 1: Tiempos de Ejecución - Cuartetos de Números Primos

5. Informe de Resultados

A partir de la experimentación, se observa una reducción drástica del tiempo de ejecución en comparación con las 31 horas del algoritmo original.

Al observar la curva en el gráfico generado, se puede notar que el tiempo de ejecución crece de manera sub-cuadrática, y se asemeja al comportamiento teórico de la función $f(N) = N\sqrt{N}$. El crecimiento no es puramente lineal, ya que a medida que los números analizados crecen, la función `esprimo(n)` debe realizar más verificaciones en su ciclo `while` (el límite de iteración es $\sqrt{N}$, que aumenta de forma monótona).

En conclusión, **los resultados empíricos se corresponden estrechamente con la complejidad temporal teórica determinada en el análisis**. La combinación de la poda del espacio de búsqueda para avanzar de a decenas y la optimización radical de la comprobación de primalidad (límite $\sqrt{N}$ y *early return*) permitieron transformar un algoritmo ineficiente de $\mathcal{O}(M^2)$ en una solución sumamente veloz de $\mathcal{O}(M\sqrt{M})$, ejecutándose en fracciones de segundo y cumpliendo sobradamente los requerimientos de eficiencia pedidos.